



Section 22 / 24

Shellcoding Techniques

As we have seen in the previous section, our `Hello World` assembly code had to be modified to produce a working shellcode. So, in this section, we'll go through some of the techniques and tricks we can use to work around any issues found in our assembly code.

Shellcoding Requirements

As we briefly mentioned in the previous section, not all binaries give working shellcodes that can be loaded directly to the memory and run. This is because there are specific requirements a shellcode must meet. Otherwise, it won't be properly disassembled on runtime into its correct assembly instructions.

To better understand this, let's try to disassemble the shellcode we extracted in the previous section from the `Hello World` program, using the same `pwn disasm` tool we previously used:

```
$ pwn disasm
'48be0020400000000000bf01000000ba12000000b801000000f05b83c000000bf000000000f05' -c
'amd64'
0: 48 be 00 20 40 00 00      movabs rsi, 0x402000
7: 00 00 00
a: bf 01 00 00 00          mov edi, 0x1
```

```

f:    ba 12 00 00 00    mov     edx, 0x12
14:    b8 01 00 00 00    mov     eax, 0x1
19:    0f 05             syscall
1b:    b8 3c 00 00 00    mov     eax, 0x3c
20:    bf 00 00 00 00    mov     edi, 0x0
25:    0f 05             syscall

```

We can see that the instructions are relatively similar to the `Hello World` assembly code we had before, but they are not identical. We see that there's an empty line of instructions, which could potentially break the code. Furthermore, our `Hello World` string is nowhere to be seen. We also see many red `00`'s, which we'll get into in a bit.

This is what will happen if our assembly code is not `shellcode compliant` and does not meet the `Shellcoding Requirements`. To be able to produce a working shellcode, there are three main `Shellcoding Requirements` our assembly code must meet:

1. Does not contain variables
2. Does not refer to direct memory addresses
3. Does not contain any NULL bytes `00`

So, let's start with the `Hello World` program we saw in the previous section, and go through each of the above points and fix them:

```

global _start

section .data
    message db "Hello HTB Academy!"

section .text
_start:

```

nasm

```
mov rsi, message
mov rdi, 1
mov rdx, 18
mov rax, 1
syscall

mov rax, 60
mov rdi, 0
syscall
```

Remove Variables

A shellcode is expected to be directly executable once loaded into memory, without loading data from other memory segments, like `.data` or `.bss`. This is because the `text` memory segments are not `writable`, so we cannot write any variables. In contrast, the `data` segment is not executable, so we cannot write executable code.

So, to execute our shellcode, we must load it in the `text` memory segment and lose the ability to write any variables. Hence, our entire shellcode must be under `'.text'` in the assembly code.

Note: Some older shellcoding techniques (like the `jmp-call-pop` technique) no longer work with modern memory protections, as many of them rely on writing variables to the `text` memory segment, which, as we just discussed, is no longer possible.

There are many techniques we can use to avoid using variables, like:

1. Moving immediate strings to registers
2. Pushing strings to the Stack, and then use them

In the above code, we may move our string to `rsi`, as follows:

```
mov rsi, 'Academy!'
```

nasm

However, a 64-bit register can only hold 8 bytes, which may not be enough for larger strings. So, our other option is to rely on the Stack by pushing our string 8-bytes at a time (in reverse order), and then using `rsp` as our string pointer, as follows:

```
push 'y!'
push 'B Academ'
push 'Hello HT'
mov rsi, rsp
```

nasm

However, this would exceed the `allowed bounds of immediate strings` `push`, which is a `dword` (4-bytes) at a time. So, we will instead move our string to `rbx`, and then push `rbx` to the Stack, as follows:

```
mov rbx, 'y!'
push rbx
mov rbx, 'B Academ'
push rbx
mov rbx, 'Hello HT'
push rbx
mov rsi, rsp
```

nasm

Note: Whenever we push a string to the stack, we have to push a `00` before it to terminate the string. However, we don't have to worry about that in this case, since we can specify the print length for the `write` syscall.

We can now apply these changes to our code, assemble it and run it to see if it works:

```
kimvutrinh@htb[/htb]$ ./assembler.sh helloworld.s
```

```
Hello HTB Academy!
```

We see that it works as expected, without needing to use any variables. We can check it with `gdb` to see how it looks at the breakpoint:

```
$ gdb -q ./helloworld
```

```
registers — $rax : 0x1
```

```
$rbx : 0x5448206f6c6c6548 ("Hello HT"?)
```

```
$rcx : 0x0
```

```
$rdx : 0x12
```

```
$rsp : 0x00007fffffffe3b8 → "Hello HTB Academy!" $rbp : 0x0
```

```
$rsi : 0x00007fffffffe3b8 → "Hello HTB Academy!" $rdi : 0x1
```

```
stack — 0x00007fffffffe3b8|+0x0000: "Hello HTB Academy!" ← $rsp,  
$rsi0x00007fffffffe3c0|+0x0008: "B Academy!"0x00007fffffffe3c8|+0x0010:  
0x00000000000002179 ("y!"?)
```

```
code:x86:64 —→ 0x40102e <_start+46> syscall
```

As we can notice, the string was built up gradually in the Stack, and when we moved `rsp` to `rsi` it contained our entire string.

Remove Addresses

We are now not using any addresses in our above code since we removed the only address reference when we removed our only variable. However, we may see references in many cases, especially with `calls` or `loops` and such. So, we must ensure that our shellcode will know how to make the call with whatever environment it runs in.

To be able to do so, we cannot reference direct memory address (i.e. `call 0xfffffffffaa8a25ff`), and instead only make calls to labels (i.e. `call loopFib`) or relative memory addresses (i.e., `call 0x401020`). We discussed rip-relative addressing in the `gdb` section.

Luckily, throughout this module, we have only been making `calls` to labels to ensure that we learn how to write code that is easily shellcoded. If we are making a `call` to a label, `nasm` will automatically change this label into a relative address, which should work with shellcodes.

If we ever had any calls or references to direct memory addresses, we can fix that by:

1. Replacing with calls to labels or rip-relative addresses (for `calls` and `loops`)
2. Push to the Stack and use `rsp` as the address (for `mov` and other assembly instructions)

If we are efficient while writing our assembly code, we may not have to fix these types of issues.

Remove NULL

NULL characters (or `0x00`) are used as string terminators in assembly and machine code, and so if they are encountered, they will cause issues and may lead the program to terminate early. So, we must ensure that our shellcode does not contain any NULL bytes `00`. If we go back to our `Hello World` shellcode disassembly, we noticed many red `00` in it:

```
$ pwn disasm
'48be0020400000000000bf01000000ba12000000b801000000f05b83c000000bf000000000f05' -c
'amd64'
0: 48 be 00 20 40 00 00      movabs rsi, 0x402000
7: 00 00 00
a: bf 01 00 00 00          mov edi, 0x1
f: ba 12 00 00 00          mov edx, 0x12
14: b8 01 00 00 00          mov eax, 0x1
19: 0f 05                  syscall
1b: b8 3c 00 00 00          mov eax, 0x3c
20: bf 00 00 00 00          mov edi, 0x0
25: 0f 05                  syscall
```

This commonly happens when moving a small integer into a large register, so the integer gets padded with an extra `00` to fit the larger register's size.

For example, in our code above, when we use `mov rax, 1`, it will be moving `00 00 00 01` into `rax`, such that the number size would match the register size. We can see this when we assemble the above instruction:

```
kimvutrinh@htb[/htb]$ pwn asm 'mov rax, 1' -c 'amd64'

48c7c001000000
```

shellsession

To avoid having these NULL bytes, we must use registers that match our data size. For the previous example, we can use the more efficient instruction `mov al, 1`, as we have been learning throughout the module. However, before we do so, we must first zero out the `rax` register with `xor rax, rax`, to ensure our data does not get mixed with older data. Let's see the shellcode for both of these instructions:

```
shellsession
kimvutrinh@htb[/htb]$ pwn asm 'xor rax, rax' -c 'amd64'

4831c0
$ pwn asm 'mov al, 1' -c 'amd64'

b001
```

As we can see, not only does our new shellcode not contain any NULL bytes, but it is also shorter, which is a very desired thing in shellcodes.

We can start with the new instruction we added earlier, `mov rbx, 'y!'`. We see that this instruction is moving 2-bytes into an 8-byte register. So, to fix it, we will first zero-out `rbx`, and then use the 2-byte (i.e. 16-bit) register `bx`, as follows:

```
nasim
xor rbx, rbx
mov bx, 'y!'
```

These new instructions should not contain any NULL bytes in their shellcode. Let's apply the same to the rest of our code, as follows:

```
xor rax, rax
mov al, 1
xor rdi, rdi
mov dil, 1
xor rdx, rdx
mov dl, 18
syscall
```

```
xor rax, rax
add al, 60
xor dil, dil
syscall
```

We see that we applied this technique in three places and used the 8-bit register for each.

Tip: If we ever need to move `0` to a register, we can zero-out that register, like we did for `rdi` above. Likewise, if we even need to `push 0` to the stack (e.g. for String Termination) we can zero-out any register, and then push that register to the stack.

If we apply all of the above, we should have the following assembly code:

```
global _start

section .text
_start:
    xor rbx, rbx
    mov bx, 'y!'
    push rbx
    mov rbx, 'B Academ'
```

```
push rbx
mov rbx, 'Hello HT'
push rbx
mov rsi, rsp
xor rax, rax
mov al, 1
xor rdi, rdi
mov dil, 1
xor rdx, rdx
mov dl, 18
syscall

xor rax, rax
add al, 60
xor dil, dil
syscall
```

Finally, We can assemble our code and run it:

```
kimvutrinh@htb[/htb]$ ./assembler.sh helloworld.s
```

```
Hello HTB Academy!
```

As we can see, our code works as expected.

Shellcoding

We can now try to extract the shellcode of our new `helloworld` program, using our previous `shellcoder.py` script:

```
shellsession
kimvutrinh@htb[/htb]$ python3 shellcoder.py helloworld

4831db66bb79215348bb422041636164656d5348bb48656c6c6f204854534889e64831c0b0014831ff40b7
```

This shellcode looks much better. But does it contain any NULL bytes? Difficult to tell. So, let's add the following line at the end of `shellcoder.py`, which would tell us if our code contains any NULL bytes and also tells us the size of our shellcode:

```
python
    print("%d bytes - Found NULL byte" % len(shellcode)) if [i for i in shellcode if
i == 0] else print("%d bytes - No NULL bytes" % len(shellcode))
```

Let's run our updated script, to see if our shellcode contains any NULL bytes:

```
shellsession
kimvutrinh@htb[/htb]$ python3 shellcoder.py helloworld

4831db66bb79215348bb422041636164656d5348bb48656c6c6f204854534889e64831c0b0014831ff40b7
61 bytes - No NULL bytes
```

As we can see, the `No NULL bytes` tells us that our shellcode is `NULL-byte free`.

Try running the script on the previous `Hello World` program to see whether it did contain any NULL bytes. Finally, we reach the moment of truth, and try to run our shellcode with our `loader.py` script to see if it runs successfully:

```
shellsession

kimvutrinh@htb[/htb]$ python3 loader.py
'4831db66bb79215348bb422041636164656d5348bb48656c6c6f204854534889e64831c0b0014831ff40t

Hello HTB Academy!
```

As we can see, we have successfully created a working shellcode for our `Hello World` program.

Connect to HTB

**Pwnbox**
Your own web-based Parrot Linux instance to play our labs.

 Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK 

Start Pwnbox

∞ spawns left

• Offline

← Previous

Section 22 / 24

XP +20

Next

Mark Complete & Next